

Introductions to Big Data Systems: Homework 5

Francisco Diego Ariel Romero Contreras

法兰西思科

2025403471

francisco.romero.c@ug.uchile.cl

Part 1

Q1. The result of mapping are shuffled and sent to reducers, which could cost a lot of network traffic. Propose an approach to address this issue.

A1.

If there is a high cost on the network traffic we can make some optimization in some cases. If the reduce is a commutative and associative operation, the mapper site can make a partial merging using a combiner function before sending to the reducer sites the key-values semi-done, which cost less network traffic.

Q2. If a mapper or reducer task is too slow, which would make the whole map-reduce task very slow. Propose an approach to address this issue.

A2.

The process of send to a mapper site to a reducer site is called “straggler”, a straggler can be slow for different reason, for example a malfunction of the memory. This problem can be solved identifying the different stragglers and when the majority of them were done start a double request on the stragglers not finished in another sites, with this the system can achieve a good time of execution without a large amount of cost.

Part 2

Introduction

Was issued a map reduce implementation for a out going vertex count. The input has the following format:

```
a 1 2 w
a 3 4 w
a 5 1 w
a 2 4 w
a 4 5 w
a 2 5 w
a 2 3 w
a 3 2 w
```

Where “a” is place holders to identify the format, we can say that the input format is like:

a Source Target weight

So, one approach that we can take is to write one at the context associated to the key *Source*. Later the reducer only has to sum by key.

```

private Text node = new Text();
public void map(Object key, Text value, Context context) throws IOException,
InterruptedException {
    StringTokenizer itr = new StringTokenizer(value.toString());
    if (itr.countTokens() >= 3) {/"a source target 0"
        itr.nextToken();// a
        String source = itr.nextToken();// source
        node.set(source);
        context.write(node, one);
    }
}

public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}

```

To achieve a correct result the input must be upload to the Hadoop environment, for simplicity let's upload the input data inside the folder "hw5_data". To upload *hw5_data* from the local directory we write:

```
/hadoop/bin/hdfs dfs -put hw5_data .
```

To run the job we type on terminal:

```
./run_od.sh
```

This will perform the map-reduce with input *hw5_data/case1* and *hw5_data/case2* at the Hadoop environment, also it create a folder *od_res* with the results on the Hadoop environment and the local directory.

The content of *run_od.sh* is:

```
#!/bin/bash
```

```
OUTPUTPATH="od_res/"
```

```
echo ===== OutDegree JAVA VERSION =====
```

```
echo ===== Compile =====
```

```
javac -classpath `/hadoop/bin/yarn classpath` OutDegree.java
```

```
jar cf wc.jar OutDegree*.class
```

```
echo
```

```
echo ===== Clear old output files on HDFS =====
```

```
/hadoop/bin/hdfs dfs -rm -r $OUTPUTPATH
```

```
echo
```

```
echo ===== Clear old output files on local =====
```

```
rm -rf $OUTPUTPATH
```

```
echo
```

```
echo ===== RUN CASE1=====
```

```
/hadoop/bin/yarn jar wc.jar OutDegree /hw5_data/case1 $OUTPUTPATH"case1"
```

```
echo
```

```
echo ===== RUN CASE2=====
```

```
/hadoop/bin/yarn jar wc.jar OutDegree /hw5_data/case2 $OUTPUTPATH"case2"
```

```
echo
```

```
echo ===== COPY output from HDFS to local =====
```

```
mkdir $OUTPUTPATH
```

```
/hadoop/bin/hdfs dfs -get $OUTPUTPATH $OUTPUTPATH
```

```
echo
```

```
echo DONE!
```

Part 3

To make the top 20 for *case2* and top2 for *case1* first we run the bash code of part 2, later we sort by count and get the head for each case.

An important thing to notice is that this solution only make the output file in local, not in the Hadoop environment.

```
#!/bin/bash
```

```
OUTPUTPATH="od_res"
```

```
source ./run_od.sh
```

```
# ===== DISPLAY RESULTS =====
```

```
echo "===== CASE1: Top 2 biggest out-degree vertices ====="
```

```
cat ./$OUTPUTPATH/case1/part-r-00000 | sort -k2 -nr | head -2 | tee "$OUTPUTPATH"/  
case1/top_case1.txt
```

```
echo
```

```
echo
```

```
echo "===== CASE2: Top 20 biggest out-degree vertices ====="
```

```
cat ./$OUTPUTPATH/case2/part-r-00000 | sort -k2 -nr | head -20 | tee "$OUTPUTPATH"/  
case2/top_case2.txt
```

```
echo
```

```
echo
```